



APACHE SPARK

Architecture Explained — Interview Prep Guide

Driver

Executors

Cluster Manager

How a Spark application actually runs across a cluster

1 What Is Apache Spark?

Apache Spark is an open-source, distributed computing engine for processing large-scale data fast, in memory, across a cluster of machines. It's the go-to answer when a dataset is too big — or a computation too slow — to handle on a single machine with tools like pandas.

Why Spark Exists (The One-Liner Interview Answer)

ELEVATOR PITCH

Spark takes a big job, splits it into small pieces, sends those pieces to run in parallel on many machines, and keeps intermediate data in memory instead of writing to disk between steps — which is why it's typically 10–100x faster than older tools like Hadoop MapReduce.

Key Characteristics

- ✓ **In-memory computation** — avoids repeated disk I/O between processing steps
- ✓ **Distributed** — work is split across many machines (nodes) in a cluster
- ✓ **Lazy evaluation** — transformations aren't run immediately; Spark builds a plan first, then executes it when an action is called
- ✓ **Fault tolerant** — lost data can be recomputed using lineage (RDD DAG), no need to restart the whole job
- ✓ **Unified engine** — one engine powers SQL, streaming, machine learning (MLlib), and graph processing (GraphX)
- ✓ **Language support** — Scala, Python (PySpark), Java, R, and SQL

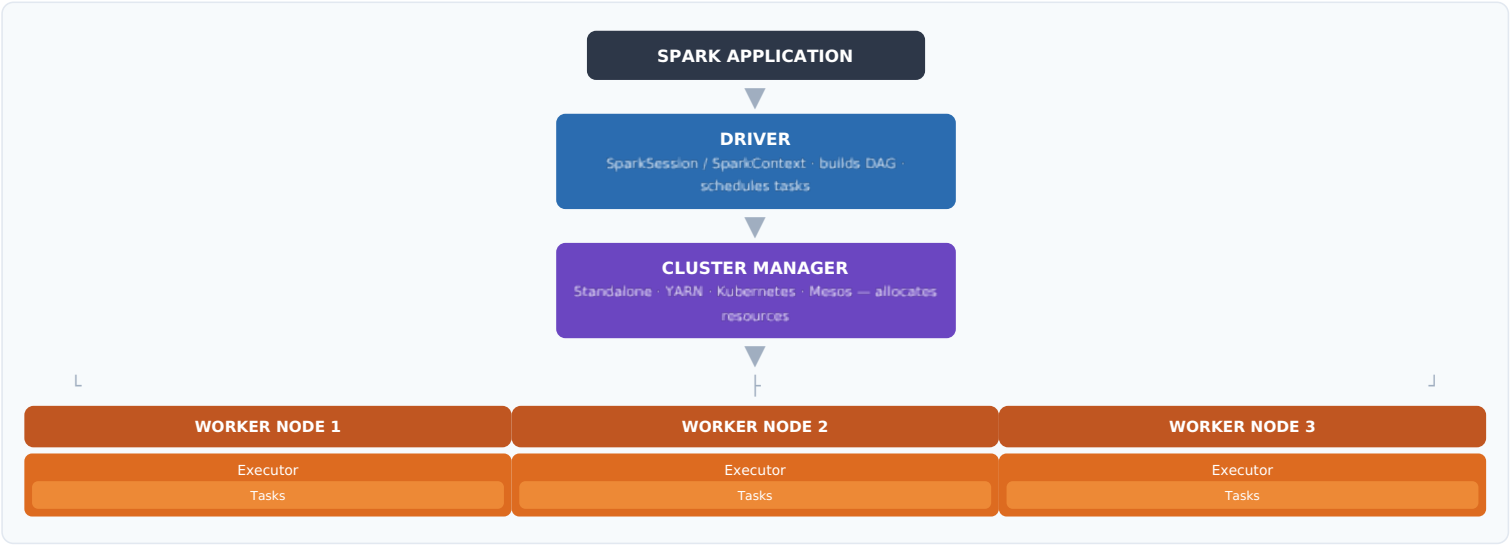
Spark vs Hadoop MapReduce (Common Interview Question)

Aspect	Hadoop MapReduce
Processing	Disk-based between each map/reduce step
Speed	Slower — disk I/O heavy
Ease of use	Verbose Java API
Use cases	Batch processing only

Apache Spark
In-memory between steps
Typically 10–100x faster for iterative workloads
Concise APIs in Python, Scala, SQL
Batch, streaming, ML, graph — all in one engine

2 Spark Architecture — The Big Picture

Every Spark application follows the same master-worker pattern: a **Driver** coordinates the work, a **Cluster Manager** allocates resources, and **Executors** on **Worker Nodes** actually run the tasks.



Component Responsibilities

Component	Main Responsibility
Driver	Runs your <code>main()</code> , creates the <code>SparkSession</code> , converts your code into a DAG of stages/tasks, and schedules work on executors
Cluster Manager	Negotiates and allocates resources (CPU, memory) for the application across the cluster
Worker Node	A physical or virtual machine in the cluster that hosts one or more executors
Executor	A JVM process on a worker node that runs tasks and holds data in memory/disk for that application
Task	The smallest unit of work — one task processes one partition of data

3 The Driver, in Detail

What the Driver Actually Does

- ✓ Hosts the **SparkSession** (and internally, the **SparkContext**) — the entry point to any Spark functionality
- ✓ Converts your DataFrame/RDD operations into a **logical plan**, then a **physical plan** (via the Catalyst optimizer)
- ✓ Splits the physical plan into a **DAG (Directed Acyclic Graph)** of **stages**
- ✓ Breaks each stage into **tasks** — one per data partition — and sends them to executors
- ✓ Tracks task progress, handles failures, and collects results back if needed (e.g. `.collect()`)

COMMON PITFALL

If you call `.collect()` on a huge DataFrame, all that data gets pulled back to the Driver's memory — which can crash the Driver even if the cluster itself has plenty of capacity. This is a favorite interview trap question.

Jobs → Stages → Tasks

Unit	What Triggers It
Job	Created every time an action is called (e.g. <code>.count()</code> , <code>.show()</code> , <code>.write()</code>)
Stage	A job is split into stages at each shuffle boundary (e.g. a <code>groupBy</code> or <code>join</code>)
Task	Each stage is split into tasks — one task per partition, run in parallel across executors

INTERVIEW PHRASE TO REMEMBER

"Transformations are lazy and build up a DAG; actions trigger execution." `map`, `filter`, `select`, `withColumn` are transformations. `collect`, `count`, `show`, `write` are actions.

4 Cluster Manager, Workers & Executors

Cluster Manager Options

- Standalone
- YARN (Hadoop)
- Kubernetes
- Apache Mesos
- Local mode (single machine)

The cluster manager's only job is **resource allocation** — deciding which worker nodes have capacity and assigning executor slots to the Driver's request. Spark itself doesn't care which one you use; the same application code runs unchanged.

Executors

- ✓ One executor is launched per application, per worker node (typically)
- ✓ Each executor runs many tasks **concurrently**, in separate threads
- ✓ Executors **cache data in memory** across operations — this is central to Spark's speed advantage
- ✓ Executors live for the **lifetime of the application**, not just one job
- ✓ If an executor fails, its tasks are rescheduled on another executor using RDD **lineage** to recompute lost data

Deployment Modes: Client vs Cluster

Mode	Where the Driver Runs
Client mode	Driver runs on the machine that launched the app (e.g. your laptop or an edge node) — good for interactive/debug sessions
Cluster mode	Driver runs inside the cluster itself, managed by the cluster manager — used for production jobs since it's more resilient

5 Execution Flow — A Worked Example

Walking through what happens end-to-end for a simple PySpark job is one of the best ways to demonstrate architecture understanding in an interview.

EXAMPLE CODE

```
from pyspark.sql import SparkSession

spark = SparkSession.builder.appName("Demo").getOrCreate()

df = spark.read.csv("sales.csv", header=True)
result = (df.filter(df.amount > 100)
          .groupBy("region")
          .count())
result.show() # action — triggers real execution
```

Step by Step

Step	What Happens
1	Driver starts, creates the SparkSession, requests executors from the Cluster Manager
2	read, filter, groupBy are transformations — Driver just builds the logical plan, no data moves yet
3	.show() is an action — Driver optimizes the plan (Catalyst), builds a DAG, splits it into stages at the shuffle caused by groupBy
4	Driver breaks each stage into tasks (one per partition) and sends them to executors
5	Executors read their partition of sales.csv, filter it locally, then shuffle data across the network to group by region
6	Final counts are computed on executors and a small result is sent back to the Driver, which prints it

6 Rapid-Fire Interview Q&A

Q What's the difference between the Driver and an Executor?

The Driver plans and coordinates the whole application (one per app). Executors actually run the tasks and hold data (many per app, spread across worker nodes).

Q What is a SparkContext vs a SparkSession?

SparkContext is the original entry point to Spark's core (RDD) functionality. SparkSession (introduced in Spark 2.0) wraps SparkContext plus SQLContext/HiveContext into a single unified entry point for DataFrames, SQL, and streaming.

Q What triggers a shuffle, and why is it expensive?

Operations like groupBy, join, and distinct require moving data between partitions/executors across the network, which involves disk I/O, serialization, and network transfer — making it the most expensive operation type in Spark.

Q How does Spark achieve fault tolerance?

Through RDD lineage — Spark remembers the sequence of transformations used to build each RDD/partition. If a partition is lost (e.g. executor crash), Spark recomputes just that partition from lineage instead of restarting the whole job.

Q What is a partition, and why does the number of partitions matter?

A partition is a chunk of the dataset that fits on one machine and is processed by one task. Too few partitions underuses the cluster (limited parallelism); too many creates scheduling overhead. This is a key performance tuning lever.

Q What does "lazy evaluation" mean and why does Spark use it?

Transformations don't execute immediately — Spark just records them. This lets the Catalyst optimizer see the whole chain of operations and build the most efficient physical plan before running anything, rather than executing each step naively.

Q Client mode vs cluster mode — when would you use each?

Client mode (Driver on the launching machine) is useful for interactive development and debugging. Cluster mode (Driver inside the cluster) is preferred for production jobs because the Driver is managed and restarted by the cluster manager if it fails.

Q What's the difference between a Job, a Stage, and a Task?

A Job is created per action. A Job splits into Stages at shuffle boundaries. A Stage splits into Tasks, one per partition, which run in parallel on executors.

FINAL TIP

If you're asked to "draw the Spark architecture" in an interview, sketch exactly this: Application → Driver (SparkSession) → Cluster Manager → Worker Nodes, each with an Executor running Tasks. Narrate it top to bottom and you'll cover 90% of what's expected.

✂ End of guide — good luck with the interview!